SIMATS ENGINEERING
ASSESSMENT TOOL 3

COURSE NAME: Artificial Intelligence **COURSE CODE:** CSA17

COURSE OUTCOME COVERED

CO2: *Experiment search and game-solving techniques such as Greedy Search, A*, CSP, Minimax, and Alpha-Beta pruning with heuristic functions for solving complex AI problems efficiently. (BL3)*

Assessment Tool Weightage: 40%

QUESTIONS: 3

Duration: 1 Hour
Total Marks: 30

Annexure A

Dry Run Evaluation

Code of Conduct:

I Mohammed Falah Miya / 1911260265 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: _____

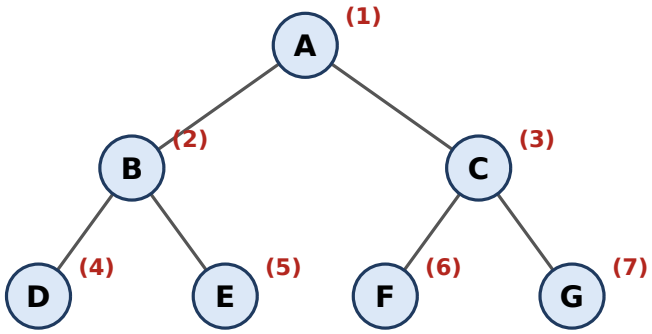
Student Name	Mohammed Falah Miya
Registration No.	1911260265
Course	Artificial Intelligence (CSA17)

Dry Run Evaluation – Answers (BL3: Apply)

CO2 is BL3 (Apply), so each answer applies the algorithm step by step, keeps the queue or stack exactly as the rules require, and gives a justified result.

Question 1 – Breadth-First Search (BFS)

Problem and strategy: Graph with A at the root, B and C as its children, D and E under B, F and G under C. Start node = A. BFS uses a **queue (FIFO)**: remove the node at the front, mark it visited, then add its unvisited children (left to right) at the rear. The number in brackets beside each node shows its visiting order.



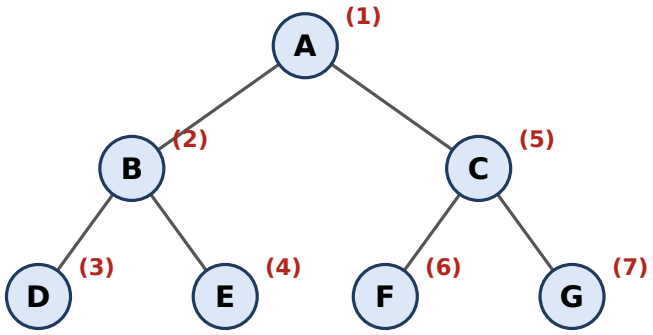
Iteration	Queue (front → rear) at start	Visited Node	Queue after adding children
1	[A]	A	[B, C]
2	[B, C]	B	[C, D, E]
3	[C, D, E]	C	[D, E, F, G]
4	[D, E, F, G]	D	[E, F, G]
5	[E, F, G]	E	[F, G]
6	[F, G]	F	[G]
7	[G]	G	[ ] (empty)

1. BFS traversal order: A → B → C → D → E → F → G (level by level: A | B, C | D, E, F, G).

2. Queue contents after each iteration: [B, C] → [C, D, E] → [D, E, F, G] → [E, F, G] → [F, G] → [G] → empty (the last column of the table).

Question 2 – Depth-First Search (DFS)

Problem and strategy: Same graph, start at A. DFS uses a **stack (LIFO)**: pop the top node, mark it visited, then push its children. To visit the left child first, children are pushed right-to-left (the left child ends up on top). The stack is written with the **top on the left**.



Iteration	Stack (top → bottom) at start	Visited Node	Stack after pushing children
1	[A]	A	[B, C]
2	[B, C]	B	[D, E, C]
3	[D, E, C]	D	[E, C]
4	[E, C]	E	[C]
5	[C]	C	[F, G]
6	[F, G]	F	[G]
7	[G]	G	[ ] (empty)

DFS traversal order: A → B → D → E → C → F → G. DFS goes as deep as possible (A → B → D), backtracks to B to take E, then returns to C and explores F and G. It visits all 7 nodes, the same as BFS, but in a different order.

Question 3 – BFS on the 8-Puzzle

Problem and strategy: The 3×3 puzzle is searched with BFS. *Initial state* and *goal state* are shown below. *Operators* are moves of the blank tile, always tried in the order **Up, Down, Left, Right** (moves that leave the board are skipped). *Cost* is 1 per move. BFS uses a FIFO queue and a **visited list**, so a state that was already generated is never added again.

Initial state

1	3	6
5		2
4	7	8

Goal state

1	2	3
4	5	6
7	8	

Checks before searching: the initial state has 6 inversions (even), so it is solvable. The sum of Manhattan distances is 8, so at least 8 moves are needed.

Naming: states are numbered **S0, S1, S2 …** in the order they are generated. S0 is the initial state. Boards are read row by row.

Dry-run table (first 5 iterations)

Iteration	Current state (expanded)	Queue before expansion	Queue after expansion	States generated
1	S0 (start)	[S0]	[S1, S2, S3, S4]	S1 (Up), S2 (Down), S3 (Left), S4 (Right)
2	S1 (Up)	[S1, S2, S3, S4]	[S2, S3, S4, S5, S6]	S5 (Left), S6 (Right)
3	S2 (Down)	[S2, S3, S4, S5, S6]	[S3, S4, S5, S6, S7, S8]	S7 (Left), S8 (Right)
4	S3 (Left)	[S3, S4, S5, S6, S7, S8]	[S4, S5, S6, S7, S8, S9, S10]	S9 (Up), S10 (Down)
5	S4 (Right)	[S4, S5, S6, S7, S8, S9, S10]	[S5, S6, S7, S8, S9, S10, S11, S12]	S11 (Up), S12 (Down)

The same steps continue until the goal state is removed from the queue. The states S0–S12 used above are:

S0

1	3	6
5		2
4	7	8

S1 (Up)

1		6
5	3	2
4	7	8

S2 (Down)

1	3	6
5	7	2
4		8

S3 (Left)

1	3	6
	5	2
4	7	8

S4 (Right)

1	3	6
5	2	
4	7	8

S5 (Left)

	1	6
5	3	2
4	7	8

S6 (Right)

1	6	
5	3	2
4	7	8

S7 (Left)

1	3	6
5	7	2
	4	8

S8 (Right)

1	3	6
5	7	2
4	8	

S9 (Up)

	3	6
1	5	2
4	7	8

S10 (Down)

1	3	6
4	5	2
	7	8

S11 (Up)

1	3	
5	2	6
4	7	8

S12 (Down)

1	3	6
5	2	8
4	7	

Level-wise summary of the complete run

Depth (moves from start)	0	1	2	3	4	5	6	7	8	9
States generated	1	4	8	8	16	32	60	72	136	163

Depths 0–7 contain 1+4+8+8+16+32+60+72 = 201 states, all expanded first. The goal is the 110th state removed at depth 8, so it is removed at **iteration 311**. The 163 states at depth 9 were generated while depth-8 states were being expanded.

Answers

1. Which node is expanded in each iteration? Iteration 1 expands S0 (initial state), iteration 2 expands S1, iteration 3 expands S2, iteration 4 expands S3 and iteration 5 expands S4 (see the table). In general BFS expands states in the order they were generated: the 1 state at depth 0 (iteration 1), the 4 states at depth 1 (iterations 2–5), the 8 at depth 2 (iterations 6–13), and so on, level by level, until the goal state is removed at iteration 311. So 310 states are expanded before the goal.

2. Total states generated: 500 states (the initial state plus 499 new states created during the 310 expansions), with repeated states discarded by the visited list. *Convention used:* a state is counted when it is first added to the queue, and the goal test is applied when a state is removed from the queue.

3. Solution path (8 moves of the blank): **Right → Up → Left → Down → Left → Down → Right → Right**.

Start

1	3	6
5		2
4	7	8

Step 1: blank Right

1	3	6
5	2	
4	7	8

Step 2: blank Up

1	3	
5	2	6
4	7	8

Step 3: blank Left

1		3
5	2	6
4	7	8

Step 4: blank Down

1	2	3
5		6
4	7	8

Step 5: blank Left

1	2	3
	5	6
4	7	8

Step 6: blank Down

1	2	3
4	5	6
	7	8

Step 7: blank Right

1	2	3
4	5	6
7		8

Step 8: blank Right

1	2	3
4	5	6
7	8	

The goal is reached in **8 moves**, which matches the lower bound of 8 from the Manhattan-distance check, so this is a shortest solution.

4. Why does BFS always find the shortest solution in an unweighted 8-puzzle? Every move costs the same (1), so the cost of a path equals its number of moves (its depth). BFS expands **all** states at depth d before any state at depth d+1. Therefore the first time the goal is reached, no shallower path to it can exist, because it would already have been found in an earlier level. The search is also complete, since the branching factor is finite (at most 4) and the state space is finite. The drawback is memory and time: the queue grew to hundreds of states for only 8 moves.